JC Penney Customer Review Analysis: Data Science Approach

Executive Summary

This report presents a comprehensive analysis of JCPenney's e-commerce review data, encompassing data manipulation in Python, as well as data analysis underpinned by visualisation techniques. Using data sets comprising 39,063 customer reviews across 7,982 products collected from 5,000 users, the methodology follows the standard of CRISP-DM (Cross-Industry Standard Process for Data Mining) to extract useful and actionable insights.

This analysis reveals a critical point in the customer satisfaction matrix of JCPenney: while 58.8% of written reviews received the lowest ranking (0-1), the sentiment analysis performed on the same reviews illustrates 89.8% positive sentiment. The 31.1% gap shows a clear misalignment between customer ratings and what they express in their comments, hinting at a design flaw instead of a product quality issue. The findings of this analysis showcase the imperative need to perform both qualitative and quantitative research to capture accurate customer insights, which directly impact the company's vendor relationships, inventory, and strategic investments.

1. Business Understanding

In retail, understanding customer satisfaction is critical for inventory decisions, essentially what products to stock, as well as for pricing strategy, marketing effectiveness and customer retention. The fundamental pillar is understanding how to reliably measure customer satisfaction to optimise such decisions. Arguably, star ratings (0-5) can only provide partial information, as users can apply inconsistent standards when assigning numeric values. The value of written reviews and text is the foundation of such analysis.

2. Data Understanding and Preparation

The analysis integrated five data sets: 39,063 reviews, 7,982 products, 5,000 customers, as well as two JSON-formatted files. Before analysis, the following three issues were identified during the quality assessment:

Negative Prices: 44 products displayed negative prices. Data entries were converted to positive values, recovering otherwise unusable data. This was identified via systematic data quality checks and negative values were converted into positive using abs () function.

Duplicate Customers: One customer (Username: dqft3311) appeared twice with different demographic information (DOB and State). Both records were removed to maintain integrity, lowering the dataset to 4,998.

Data Completeness: 2,166 products do not have price information. No additional duplicates were detected either in the Review or Product dataset.

3. Visualisation and Analysis

Distribution Analysis – Visual Table & Violin Plot

The analysis of the Review's dataset revealed how users rated the products that they purchased:

Rating	Count	Percentage
0-1	**22,952**	**58.8%**
2-3	**12,064**	**30.8%**
4-5	**4,147**	**10.6%**

This dataset presents a striking bimodal distribution. At first glance, this pattern indicates wide customer dissatisfaction. See Fig 1 (Violin Chart below – Review Score Distribution).

Fig. 1 – Review Score Distribution

JCPenney executives might deduce that the products they are selling are not of good quality and customer feedback is rather poor, which could lead to vendor replacements and product improvements.

Though this analysis is truly incomplete if the qualitative data is not taken into consideration.

3.1 Sentiment Analysis – Text Driven (Pie Chart)

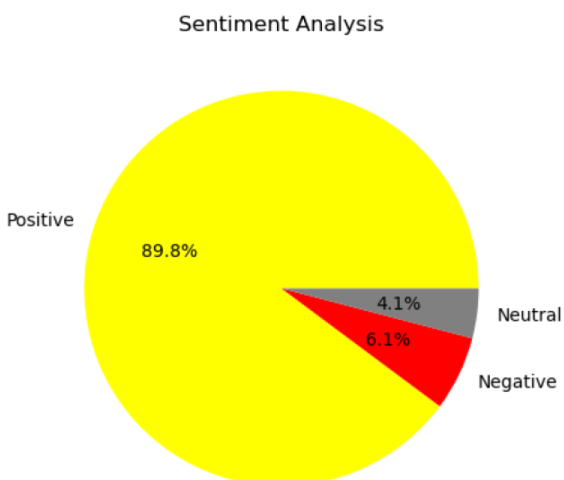
This approach was selected because of the following:

- Such analysis focuses on reviews and text specifically.
- Capture nuance, as it automatically recognises positive words, like *excellent quality* and negative approaches, such as *poor material* as well as punctuation emphasis.

Deploying VADER (Valence Aware Dictionary and sEntiment Reasoner) sentiment analysis (below Fig 2.) to study the written text reviews it appears that:

- 89.8% of the data provided can be considered as positive sentiment (35,097 Reviews)
- 4.1% present neutral sentiment (1,115 Reviews)
- Just 6.1% show negative sentiment (2,811 Reviews)

Fig. 2 – Sentiment Analysis on Reviews



It is crucial to highlight that the same dataset has been analysed by looking at it from two different angles, namely a violin plot and sentiment analysis. The exact same customers expressing low ratings (0-1) are writing positive reviews. The 31.1% gap between positive sentiment and low ratings shows a clear misalignment between customer intent and what the rating system records. If JCPenney solely relies on the 58.8% low-rating figure, the company might incorrectly remove inventory, terminate vendors' contracts and initiate campaigns to improve its products. Thanks to the sentiment analysis performed, 89.8% show positive sentiment, which validates the hypothesis that customers are satisfied.

3.2 Root Cause Analysis – Interface Hypothesis

The extreme divergence between the review scores and sentiment illustrated above implies design and presentation flaws in the submission rating system. The plausible problems could be as follows:

- Inverted Rating Scale: The star rating system may be displayed counter-intuitively, which could cause customers to click on a low rating when satisfied and wanting to leave a high score.
- Interface Scaling Problems: Since users may prefer to leave feedback on a mobile phone, they might give lower ratings due to the placement of the rating scale on mobile devices.

Analysis of the dataset shows that a 31.1% gap between low review scores and positive sentiment remains stable across every price range and product category taken into account. No irregularities relating to product quality or user behaviors have been detected in the given subgroups, which suggests that this is not an issue linked to specific products or price segments. In fact, this pattern points to a platform-wide issue affecting the review process itself. If this gap had been isolated to certain price points or products, it would indicate either price sensitivity across buyers or reluctance to provide high ratings in selected categories. Though the data visualisation above shows pronounced bimodal distribution in review scores, highlighting this gap. Two peaks in the score distribution support the conclusion that user experience is not driving this output; rather, the user interface may be responsible. Addressing this root cause on a platform level is fundamental, as target actions towards individual groups would not address this issue.

3.3 Reasons behind the two performed tests

→ **Violin Plot Analysis (Ratings Focused)**

This test shows 58.8% low ratings (0-1), which suggests a broad dissatisfaction with products and can lead to inventory cuts, false crisis, and vendor losses.

→ **Sentiment Chart (Text Focused)**

The results of this test reveal that 89.8% of the reviews in the dataset present positive sentiment, suggesting a high level of customer satisfaction with their purchases.

To sum up, by performing both analyses, namely Violin Plot Analysis and Sentiment Analysis, a disconnection gap of 31.1% has been identified. This suggests a clear front-end issue, as the ratings given by the users do not match the positive sentiment data extracted by the review data set analysed.

4. Recommendations & Business Actions

The next paragraph focuses on the recommendations for the retail company.

i. Redesign Review Interface

Firstly, given the outcomes of the data analyses above, JCPenney should revisit its review rating interface. The datasets underpin compelling evidence, especially the 31.1% gap between low scores and extremely positive review sentiment analysis, which indicates that majority of satisfied users are invertedly submitting low rating scores. JCPenney should lead live user testing sessions analysing in real time how its users interact with the rating scales on desktops and mobile devices. Such sessions, which could be benchmarked against other retail leaders like Walmart and Amazon, would highlight any issues triggered by the rating of scales, symbols, or unclear areas on the front-end platform.

ii. Root Cause Analysis

Secondly, JCPenney should establish a dedicated team to investigate the reasons behind the striking discrepancy between high customer sentiment and low ratings. This target research could focus on a sample of reviews that express positive sentiment but give low ratings, aiming to determine whether customers are downgrading their rating due to issues like packaging, shipping or other frustrations, despite being satisfied with the actual product. Adopting a root cause investigation will unveil the recurring issues and complaints that are linked with low scores.

iii. Dual-Metric Analytics

Lastly, dashboards and reports should be updated to display both qualitative sentiments as well as quantitative ratings, which would offer a dual-metric view of the company. In fact, campaigns, products, and vendors should be evaluated not only by the numerical scores obtained from user reviews, but also by the alignment (or misalignment) of data captured in the written reviews. This technique will empower JCPenney to spot areas that require adjustments, interventions or support, but also meaningful opportunities for market position. By adopting the above steps, the company can ensure that the collected data can be a strategic asset, as it provides actionable insights that can lead to improved products, higher customer satisfaction, and ultimately higher revenue.

5. Conclusion

To conclude, this data analysis revealed a critical insight that poses a risk to conventional satisfactory measurement: relying on numeric data scores can be misleading and a risky approach. Having used sentiment analysis by VADER, this report unlocked the authentic customer perceptions, as it revealed that 89.8% are satisfied users, unlike what the violin chart had initially unveiled. This paradox can be related to several flaws in the process of submitting ratings, as customers who are keen to express their satisfaction are recorded as unsatisfied due to the poor design of the front-end platform. While 27% of products lacked price information, this did not impact the sentiment analysis, as review data was complete for all 39,063 records. By implementing audits within the platform's back end and refining the feedback interface, JCPenney can minimise missed opportunities and costly errors. Through the integration of both quantitative and qualitative strategies, the company will enhance its vendor relationship management and develop a holistic understanding of its daily customers.

JC Penney E-Commerce Review Analysis

Student ID: 3541011

Objective: Identify discrepancies between qualitative sentiment and quantitative ratings

Key Findings: 31.1% gap between low ratings (58.8%) and positive sentiment (89.8%)

Datasets: 39,063 reviews across 7,982 products from 4,998 users

Data Exploration

Library Loading

In [1]:
```
import pandas as pd
import numpy as np
import json
from IPython.display import display
import matplotlib.pyplot as plt
import seaborn as sns
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
```

Data Loading

In [2]:
```
users = pd.read_csv('users.csv')
reviews = pd.read_csv('reviews.csv')
products = pd.read_csv('products.csv')
```

In [3]:
```
with open ('jcpenney_products.json') as f:
    jcpenney_products = [json.loads(line) for line in f if line.strip()]
    jcpenney_products = pd.DataFrame(jcpenney_products)

with open ('jcpenney_reviewers.json') as f:
    jcpenney_reviewers = [json.loads(line) for line in f if line.strip()]
    jcpenney_reviewers = pd.DataFrame(jcpenney_reviewers)
```

Data Quality Assessment

In [4]:
```
# Display first 5 rows of each data set to understand the structure
display(products.head())
display(reviews.head())
display(users.head())
display(jcpenney_products.head())
display(jcpenney_reviewers.head())
```

	Uniq_id	SKU	Name	Description	Price	Av_Sc
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	2.
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	2.
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.
4	d969a8542122e1331e304b09f81a83f6	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	Youll return to our Alfred Dunner pull-on capr...	41.09	3.

	Uniq_id	Username	Score	Review
0	b6c0b6bea69c722939585baeac73c13d	fsdv4141	2	You never have to worry about the fit...Alfred...
1	b6c0b6bea69c722939585baeac73c13d	krpz1113	1	Good quality fabric. Perfect fit. Washed very ...
2	b6c0b6bea69c722939585baeac73c13d	mbmg3241	2	I do not normally wear pants or capris that ha...
3	b6c0b6bea69c722939585baeac73c13d	zeqg1222	0	I love these capris! They fit true to size and...
4	b6c0b6bea69c722939585baeac73c13d	nvfn3212	3	This product is very comfortable and the fabri...

	Username	DOB	State
0	bkpn1412	31.07.1983	Oregon
1	gqjs4414	27.07.1998	Massachusetts
2	eehe1434	08.08.1950	Idaho
3	hkxj1334	03.08.1969	Florida
4	jjbd1412	26.07.2001	Georgia

	uniq_id	sku	name_title	description	list_price
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09
4	d969a8542122e1331e304b09f81a83f6	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09

	Username	DOB	State	Reviewed
0	bkpn1412	31.07.1983	Oregon	[cea76118f6a9110a893de2b7654319c0]
1	gqjs4414	27.07.1998	Massachusetts	[fa04fe6c0dd5189f54fe600838da43d3]
2	eehe1434	08.08.1950	Idaho	[]
3	hkxj1334	03.08.1969	Florida	[f129b1803f447c2b1ce43508fb822810, 3b0c9bc0be6...
4	jjbd1412	26.07.2001	Georgia	[]

In [5]:

```
products.info()
reviews.info()
users.info()
jcpenney_products.info()
jcpenney_reviewers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7982 entries, 0 to 7981
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   Uniq_id      7982 non-null   object 
 1   SKU          7915 non-null   object 
 2   Name         7982 non-null   object 
 3   Description  7439 non-null   object 
 4   Price        5816 non-null   float64
 5   Av_Score     7982 non-null   float64
dtypes: float64(2), object(4)
memory usage: 374.3+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 39063 entries, 0 to 39062
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Uniq_id   39063 non-null  object
 1   Username  39063 non-null  object
 2   Score     39063 non-null  int64 
 3   Review    39063 non-null  object
dtypes: int64(1), object(3)
memory usage: 1.2+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   Username  5000 non-null   object
 1   DOB       5000 non-null   object
 2   State     5000 non-null   object
dtypes: object(3)
memory usage: 117.3+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7982 entries, 0 to 7981
Data columns (total 15 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   uniq_id                 7982 non-null   object 
 1   sku                     7982 non-null   object 
 2   name_title              7982 non-null   object 
 3   description             7982 non-null   object 
 4   list_price              7982 non-null   object 
 5   sale_price              7982 non-null   object 
 6   category                7982 non-null   object 
 7   category_tree           7982 non-null   object 
 8   average_product_rating  7982 non-null   float64
 9   product_url             7982 non-null   object 
 10  product_image_urls      7982 non-null   object 
 11  brand                   7982 non-null   object 
 12  total_number_reviews    7982 non-null   int64  
 13  Reviews                 7982 non-null   object 
 14  Bought With             7982 non-null   object 
dtypes: float64(1), int64(1), object(13)
memory usage: 935.5+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype 
```

```
---  ------    --------------  -----
 0   Username  5000 non-null   object
 1   DOB       5000 non-null   object
 2   State     5000 non-null   object
 3   Reviewed  5000 non-null   object
dtypes: object(4)
memory usage: 156.4+ KB
```

In [6]:
```
print(products.shape)
print(reviews.shape)
print(users.shape)
print(jcpenney_products.shape)
print(jcpenney_reviewers.shape)
```

```
(7982, 6)
(39063, 4)
(5000, 3)
(7982, 15)
(5000, 4)
```

In [7]:
```
reviews = pd.read_csv('reviews.csv')
products = pd.read_csv('products.csv')

print("Reviews Columns:", reviews.columns)
print("Products columns:",products.columns)
```

```
Reviews Columns: Index(['Uniq_id', 'Username', 'Score', 'Review'], dtype='objec
t')
Products columns: Index(['Uniq_id', 'SKU', 'Name', 'Description', 'Price', 'Av_Sc
ore'], dtype='object')
```

Data Cleaning

In [26]:
```
products['Price'] = pd.to_numeric(products['Price'], errors='coerce')
products['Price'] = products['Price'].abs() #Convert all negatives to positive
print((products['Price'] < 0).sum()) #Perform a quick check
```

```
0
```

In [9]:
```
#Tabulating review scores gives an overview of customer ratings
table = reviews['Score'].value_counts().sort_index()
simple_table = table.reset_index()
simple_table.columns = ['Score', 'Nr Reviews']

print(simple_table.to_string(index=False))
```

```
 Score  Nr Reviews
     0       11265
     1       11687
     2        7957
     3        4007
     4        2245
     5        1902
```

The table above illustrates how many customer reviews were received for each score, ranging from 0 (lowest) to 5 (highest). The most common scores are 0 and 1, each with over 11,000 reviewers, while higher scores like 4 and 5 are much less frequent. This distribution suggests that customers tend to give lower ratings more often in this dataset. Understanding the breakdown of review scores helps identify overall satisfaction levels and pinpoints areas where product or service improvements might be needed.

Removing Duplicates

This line of code systematically detects all users in the dataset who share the same username, thereby identifying potential duplicate accounts and data inconsistencies. Performing this operation is essential for rigorous data cleaning as it ensures that each individual is uniquely represented, which supports the reliability and integrity of the analysis.

In [10]:
```
potential_duplicates = users[users.duplicated(subset=["Username"], keep=False)]
print(potential_duplicates)
```

```
      Username         DOB       State
731   dqft3311  28.07.1995   Tennessee
2619  dqft3311  03.08.1969  New Mexico
```

By running this code, it has been identified that some users in the dataset share the same username, namely "dqft3311", appearing with two different records. This finding suggests the presence of duplicate user accounts or potential data entry errors, which might affect the accuracy of this report.

The following line removes all records for the user "dqft3311" where the date of birth is either "28.07.1995" or "03.08.1969". Doing this ensures that potential duplicate accounts with the same username but different personal details are eliminated, ensuring data integrity.

In [11]:
```
users = users[~((users['Username'] == 'dqft3311') &
                users['DOB'].isin(['28.07.1995','03.08.1969']))]
```

Identify Duplicate Reviews and Products

In [12]:
```
if reviews.duplicated().any():
    print("There are duplicate reviews in the dataset.")
else:
    print("There are no duplicate reviews in the dataset.")
```

```
There are no duplicate reviews in the dataset.
```

In [13]:
```
if products.duplicated().any():
    print("There are duplicate rows in the products dataset.")
else:
    print("There are no duplicate rows in the products dataset.")
```

```
There are no duplicate rows in the products dataset.
```

In [14]:
```
json_duplicate = jcpenney_reviewers[jcpenney_reviewers.duplicated(subset="Userna
print(json_duplicate)
```

```
      Username         DOB       State                            Reviewed
731   dqft3311  28.07.1995   Tennessee  [5f280fb338485cfc30678998a42f0a55]
2619  dqft3311  03.08.1969  New Mexico  [571b86d307f94e9e8d7919b551c6bb52]
```

It is also necessary to remove duplicate records from JSON file to ensure that each user entry is unique.

In [15]:
```
user_filter = jcpenney_reviewers['Username'] == 'dqft3311'
dob_filter = (jcpenney_reviewers['DOB'] == '28.07.1995') | (jcpenney_reviewers['
combined_filter = user_filter & dob_filter
jcpenney_reviewers = jcpenney_reviewers[~combined_filter]
```

JSON files provide supplementary data, including product URLs, brand information, and sale prices. For this analysis, the CSV files were primarily used, which contained the essential review and rating data. JSON files were utilised to cross-validate data quality and ensure consistency across data sources.

Analysis of products with and without prices

Analysing products based on whether they have listed prices provides important insights into both catalogue completeness and potential gaps in data collection. Products without price information can signal issues such as incomplete product records.

In [16]:
```
products = pd.read_csv('products.csv')
with_prices = products['Price'].notna().sum()
missing_prices = products['Price'].isna().sum()

print(f"Products with price: {with_prices}")
print(f"Products missing price: {missing_prices}")
print(f"Total products: {len(products)}")
```

```
Products with price: 5816
Products missing price: 2166
Total products: 7982
```

Comprehensive Data Quality Checks Prior to Visualisation

Before proceeding to data visualisation, it is essential to conduct a comprehensive final validation to ensure all the steps taken above have been correctly executed. Verifying the effectiveness of these cleaning operations helps maintain the accuracy, reliability and consistency of the data and minimises the risk of producing misleading results.

In [17]:
```
print(products.shape)
print(reviews.shape)
print(users.shape)
print(jcpenney_products.shape)
print(jcpenney_reviewers.shape)
```

```
(7982, 6)
(39063, 4)
(4998, 3)
(7982, 15)
(4998, 4)
```

In [27]:
```
print("="*24, "\nFinal Data Quality Check\n" + "="*24)

missing = products['Price'].isnull().sum()
negative = (products['Price'] < 0).sum()
zero = (products['Price'] == 0).sum()

print(f"Mising prices: {missing}")
print(f"Negative prices: {negative}")
print(f"Zero prices: {zero}")

print(f"""
Min price: ${products['Price'].min():.2f}
Max price: ${products['Price'].max():.2f}
Total products: {len(products)}
""")
```

```
========================
Final Data Quality Check
========================
Mising prices: 2166
Negative prices: 0
Zero prices: 0

Min price: $8.01
Max price: $17122.17
Total products: 7982
```

Data Visualisation

To facilitate comprehensive analysis, the reviews and users datasets were merged on the 'Username' field, followed by merging this result with the products dataset on 'Uniq_id'. This process produces a single, unified DataFrame that incorporates all relevant review scores, reviewer demographics, and product attributes. By using this dataset for visualisation and statistical analysis, the findings reflect the full context of customer experiences and product characteristics, which enhances the reliability and interpretability of the results.

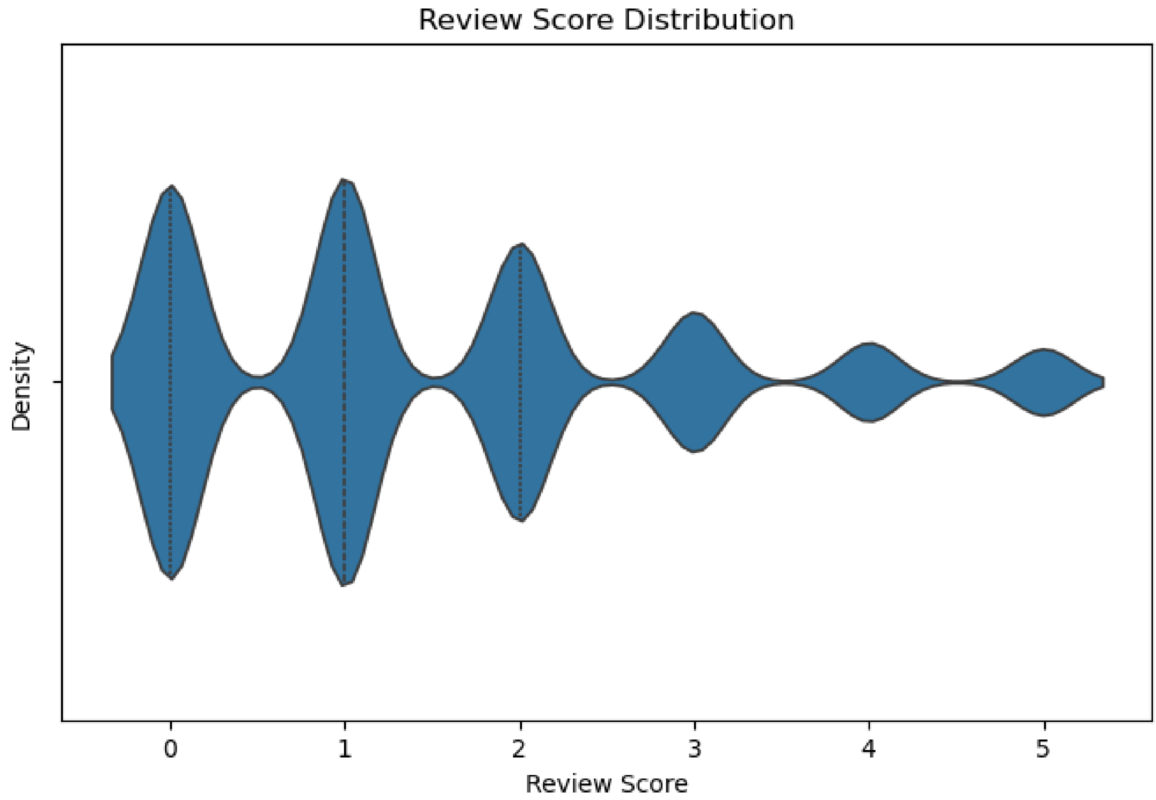
In [19]:
```
merged_reviews_users = pd.merge(reviews, users, on='Username')
merged_all = pd.merge(merged_reviews_users, products, on='Uniq_id')
```

Distribution Analysis - Violin Plot

The violin plot visualises the distribution of the review scores in the data set to understand how ratings are spread across customers. Unlike a box plot, the violin plot shows the full distribution shape and density, revealing where reviews cluster and how common different scores are. Such insights help identify whether customers tend to give mostly high, low or mixed ratings, which is crucial for assessing overall product satisfaction.

In [20]:
```
plt.figure(figsize=(8,5))
sns.violinplot(x='Score', data=merged_all, width=0.6, inner="quartile")
```

```
plt.title('Review Score Distribution')
plt.xlabel('Review Score')
plt.ylabel('Density')
plt.show()
```

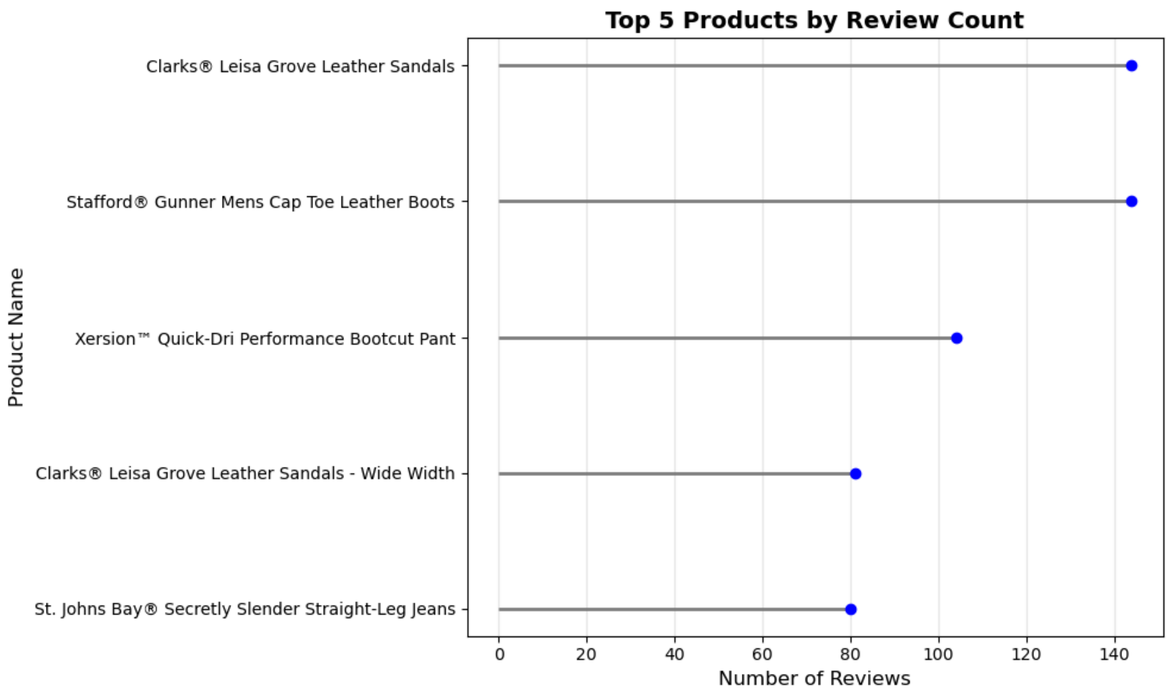


The violin plot above visually summarises the distribution of customer review scores in the dataset. The plot shows a pronounced bimodal distribution with peaks at scores 0 and 1, as they appear to be the widest, indicating that low ratings are most common. Conversely, the narrow sections at scores 4 and 5 reveal that higher ratings are rare. This analytical view clarifies how customer feedback is skewed substantially towards ratings and why further investigation into rating patterns is important for business insights.

Visualising the Most Reviewed Products

This section creates a lollipop chart to show the top five products with the highest number of user reviews. Such visualisation helps identify which products have generated the most engagement and customer feedback.

In [21]:
```
most_reviewed = merged_all["Name"].value_counts().head(5).sort_values()

plt.figure(figsize=(10,6))
plt.hlines(most_reviewed.index, 0, most_reviewed.values, color='gray', linewidth
plt.plot(most_reviewed.values, most_reviewed.index, "o", color='blue')
plt.title("Top 5 Products by Review Count", fontsize=14, fontweight='bold')
plt.ylabel("Product Name", fontsize=12)
plt.xlabel("Number of Reviews", fontsize=12)
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```



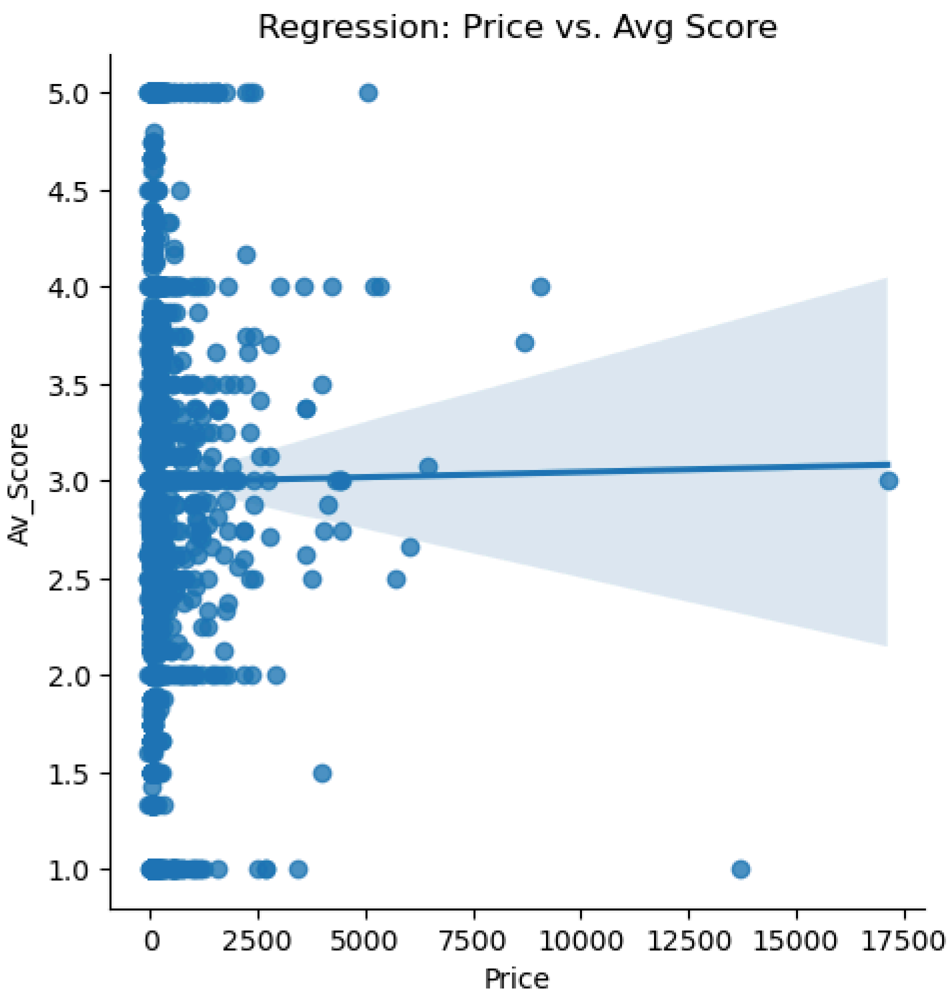
The chart above highlights the five products that received the highest number of customer reviews in the dataset given. Clarks Leisa Grove Leather Sandals had the most engagement overall, followed by Stafford Gunner Mens Cap toe Leather Boots and other popular items. This suggests that these products either have high sales, strong interest from the public or that they generate more feedback due to their marketing. Recognising the most reviewed products is essential to understand purchasing behaviours, as it guides future business strategies toward inventory that consistently attracts customer engagement.

Regressions vs. Avg Score

Before running the regression analysis, it is useful to explore if there is a relationship between product price and customer satisfaction, as measured by average review score. Correlation analysis, such as regression, is a standard technique for determining if two continuous variables are related. By visualising price alongside reviewer ratings, it can be assessed if higher-priced items receive better (or worse) feedback.

In [22]:

```
sns.lmplot(x='Price', y='Av_Score', data=products)
plt.title('Regression: Price vs. Avg Score')
plt.show()
```



This regression plot visualises the connection between product price and average customer review score in the dataset. Each point represents a product, while the blue line and shaded area show the fitted regression and confidence interval. The outcome shows no strong correlation between average review score and price. The regression is mostly flat, suggesting that items with higher price tags do not receive lower or higher ratings. The finding suggests that users do not associate higher prices with higher or lower satisfaction, as most products cluster with an average score of around 3.0.

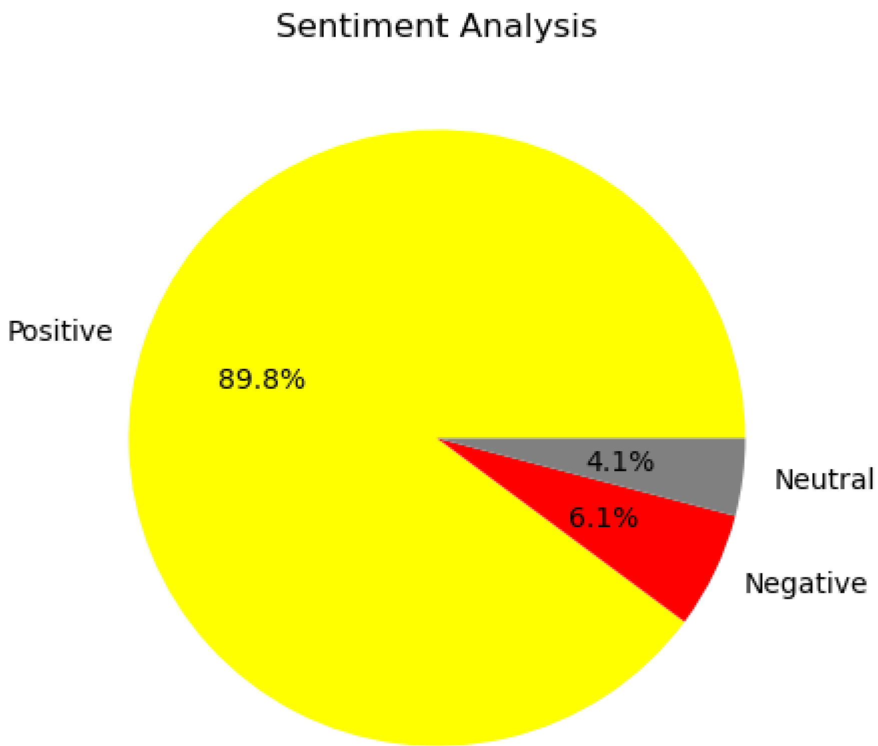
Sentiment Analysis on Reviews (VADER)

Before running the code, it is important to explain why the VADER for sentiment analysis has been selected. This test converts written feedback into measurable sentiment scores, which clearly go beyond what numeric ratings alone reveal. It will shed light on the strengths and weaknesses in customer experience.

In [23]:
```
analyzer = SentimentIntensityAnalyzer()
review_texts = reviews['Review'].astype(str)
reviews['sentiment_score'] = review_texts.apply(lambda text: analyzer.polarity_s

#Define sentiment classification using standard VADER thresholds
#Positive: compound score > 0.05
#Negative: compound score < -0.05
#Neutral: between -0.05 and 0.05
```

```python
def label_sentiment(score):
    if score > 0.05:
        return 'Positive'
    elif score < -0.05:
        return 'Negative'
    else:
        return 'Neutral'

reviews['sentiment'] = reviews['sentiment_score'].apply(label_sentiment)
sent_counts = reviews['sentiment'].value_counts()

print("Sentiment Distribution:")
print(sent_counts)
print(f"\nTotal reviews: {len(reviews):,}")

plt.figure(figsize=(8, 5))
plt.pie(sent_counts.values, labels=sent_counts.index, autopct='%1.1f%%',
        colors=['yellow', 'red', 'grey'])
plt.title(f'Sentiment Analysis')
plt.show()
```

```
Sentiment Distribution:
sentiment
Positive     35092
Negative      2385
Neutral       1586
Name: count, dtype: int64

Total reviews: 39,063
```



The pie chart above shows the overall distribution of customer review sentiment for JCPenney products. The vast majority of reviews are labelled as Positive (89.8%), with only a small portion being Negative (6.1%) and Neutral (4.1%).

In [24]:

```
#Calculate % of low ratings (score 0 or 1)
low_ratings = reviews[reviews['Score'] <= 1]
low_rating_pct = len(low_ratings) / len(reviews) * 100

#Calculate % of positive sentiment
positive_reviews = reviews[reviews['sentiment'] == 'Positive']
positive_sentiment_pct = len(positive_reviews) / len(reviews) * 100

#Calculate the gap between positive sentiment and low ratings
gap = positive_sentiment_pct - low_rating_pct

#Print
print(f"Low Ratings (0 or 1 ONLY):")
print(f"   {len(low_ratings)} reviews ({low_rating_pct:.1f}%)")
print(f"Positive Sentiment:")
print(f"   {len(positive_reviews)} reviews ({positive_sentiment_pct:.1f}%)")
print(f"Gap: {gap:.1f}% (Difference between Positive Sentiment and Low Ratings)"
#Using :.1f in f-strings displays numbers to one decimal place (rounded)
#This ensures clarity when reporting statistics such as percentages.
```

```
Low Ratings (0 or 1 ONLY):
   22952 reviews (58.8%)
Positive Sentiment:
   35092 reviews (89.8%)
Gap: 31.1% (Difference between Positive Sentiment and Low Ratings)
```

This code calculates three key metrics for the review dataset: the percentage and count of reviews with low star ratings (0 or 1), the percentage and count of reviews showing positive sentiment based on text analysis and the gap between these two values. For this data, there are 22,952 low-rated reviews (58.8%), 35,092 positive sentiment reviews (89.8%), and a gap of 31.1%. Comparing the numerical ratings to text-based sentiment reveals a clear gap of 31.1%, indicating a striking misalignment between the customer's tone and the star rating system. This suggests that most customers leave favourable feedback, reflecting high satisfaction with the products in the dataset, which is often not captured by numeric scores alone. The findings emphasise the importance of using sentiment analysis alongside star ratings to accurately understand customer satisfaction and guide business decisions.

In [28]:

```
!pip install -q nbconvert[webpdf]
!jupyter nbconvert "BD2_Assignment.ipynb" --to webpdf --allow-chromium-download
```

```
[NbConvertApp] Converting notebook BD2_Assignment.ipynb to webpdf
[NbConvertApp] WARNING | Alternative text is missing on 4 image(s).
[NbConvertApp] Building PDF
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 318039 bytes to BD2_Assignment.pdf
```